Watermelon Smart Irrigation

User Manual

ESP32 boards | Firebase database | ML model | Mobile app

This manual explains how to set up, run and use the whole system: the three ESP32 boards and their soil sensors, the Firebase database, the machine learning (ML) model and the mobile app. It also lists every pump rule, screen message and setting, and what to do when something goes wrong.

Who this is for: the person who installs, runs and checks the irrigation system in the field. No programming knowledge is needed for daily use; Sections 3 and 4 need the Arduino IDE.

Version: October 2026

Contents

1. How the System Works 3

2. Hardware and Wiring 4

3. Installing the ESP32 Programs 5

4. Calibrating the Soil Sensors 6

5. When the Pump Turns On and Off 7

6. The Main Board's Screen (OLED) 8

7. The Firebase Database 9

8. The ML Model 10

9. The Mobile App 11

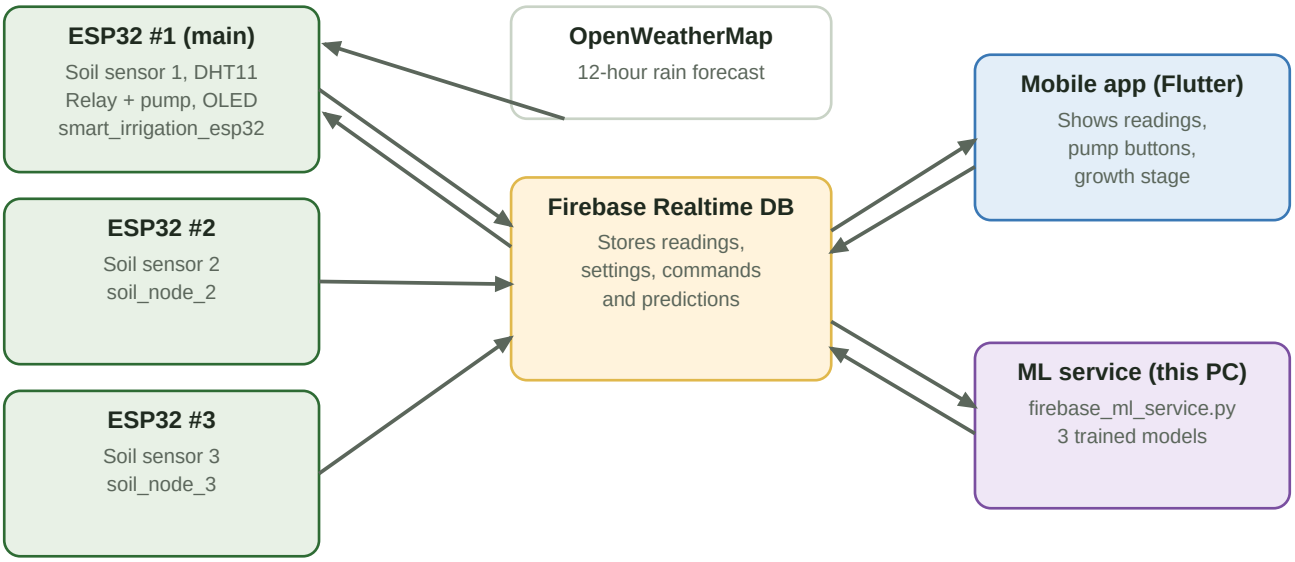
10. Daily Operation Checklist 12

11. Troubleshooting 13

12. Settings Reference 14

1. How the System Works

Three ESP32 boards each read one soil moisture sensor and send the reading over Wi-Fi to a Firebase database. The main board (#1) also measures temperature and humidity, gets a 12-hour rain forecast, and switches the water pump. The mobile app reads everything from the database and can turn the pump on or off. A Python service on this PC runs the ML model and writes an irrigation recommendation back to the database.



Part	What it does
ESP32 #1 (main)	Reads soil sensor 1, temperature and humidity; reads sensors 2 and 3 from the database; gets the rain forecast; switches the pump using the AVERAGE of the 3 sensors; shows status on the OLED screen.
ESP32 #2 and #3	Each reads one soil sensor and saves it to the database. Their readings are part of the average the pump uses.
Firebase database	Online storage in the middle. It stores readings, history, the growth stage, pump commands and predictions. It does not run any code.
ML service	A Python program on this PC. It reads the latest reading, runs 3 trained models, and writes a recommendation. It only advises; it does not switch the pump.
Mobile app	Shows readings, each sensor, the recommendation and charts; lets you set the growth stage and turn the pump on or off.

Important: Only the main ESP32 switches the pump. It uses the **average** of all 3 soil sensors. Example: 50%, 80% and 85% average 72%, so the pump stays OFF.

2. Hardware and Wiring

Parts list

Item	Qty	Notes
ESP32 DevKit V1 (30-pin)	3	One main board, two sensor boards
Capacitive soil moisture sensor v2.0	3	One per board. Wires: VCC (red), GND (black), AOUT (yellow)
DHT11 temperature/humidity sensor	1	Main board only
1-channel relay module + water pump	1	Main board only. Relay is active-LOW
SSD1306 OLED 128x64 (I2C)	1	Main board only, optional
Breadboards, jumper wires, USB cables	-	Use data USB cables, not charge-only

Main board (ESP32 #1)

Part	Part pin	ESP32 pin
Soil sensor 1	VCC / GND / AOUT	3V3 / GND / **D34**
DHT11	VCC / GND / DATA	3V3 / GND / **D4**
Relay module	VCC / GND / IN	VIN (5 V) / GND / **D26**
OLED SSD1306	VCC / GND / SDA / SCL	3V3 / GND / **D21** / **D22**

Sensor boards (ESP32 #2 and #3)

Sensor pin	ESP32 pin
VCC (red)	3V3
GND (black)	GND
AOUT (yellow)	**D34**

Wiring tips

- The soil sensor's AOUT must go to an **ADC1** pin (GPIO 32-39). ADC2 pins (GPIO 0, 2, 4, 12-15, 25-27) stop reading while Wi-Fi is on. D34 is the 5th pin from the EN end, on the same side as VP, VN and D35.
- Put each wire in the **same breadboard row** as the ESP32 pin. One row off is the most common mistake.
- All grounds must be connected together (sensor GND, relay GND, ESP32 GND).
- If the small chip on a soil sensor is marked **NE555** (not TLC555), it needs 5 V: connect its VCC to **VIN** instead of 3V3. Its output stays below 3.3 V, so D34 is safe.
- Insert the soil sensor only up to the line printed on it. Keep the electronics at the top dry.

3. Installing the ESP32 Programs

Each board runs its own sketch (.ino file). Each sketch is a folder with the same name:

Folder / sketch	Upload to
smart_irrigation_esp32 / smart_irrigation_esp32.ino	ESP32 #1 (main, with pump)
soil_node_2 / soil_node_2.ino	ESP32 #2
soil_node_3 / soil_node_3.ino	ESP32 #3
soil_pin_test / soil_pin_test.ino	Any board, only for testing wiring

One-time setup

1. Install the **Arduino IDE**. In **Boards Manager**, install **esp32 by Espressif**.
2. In **Library Manager**, install: **ArduinoJson** (version 7), **DHT sensor library** (Adafruit), **Adafruit GFX** and **Adafruit SSD1306**.
3. In each sketch folder, make a file named **secrets.h** by copying **secrets.example.h**, and type your Wi-Fi name, Wi-Fi password and OpenWeatherMap API key. All three boards must use the same Wi-Fi network, and it must be **2.4 GHz**.
4. In the main sketch, set **FARM_LAT** and **FARM_LON** to your farm's GPS position (long-press the spot in Google Maps to copy it). This is used for the rain forecast.

```
// secrets.h  (one in each sketch folder - never share or upload this file)
const char* WIFI_SSID = "your-wifi-name";
const char* WIFI_PASS = "your-wifi-password";
const char* OWM_KEY   = "your-openweathermap-api-key";
```

Uploading a sketch

1. Plug the board into the PC with a USB data cable. Close the Serial Monitor if it is open.
2. Open the .ino file. If you just changed secrets.h, close and reopen the sketch first.
3. Choose **Tools > Board > esp32 > ESP32 Dev Module**, the board's **Port** (for example COM6), and **Tools > Upload Speed > 115200**.
4. Click **Upload**. Wait for *Hash of data verified* and *Hard resetting*.
5. Open **Tools > Serial Monitor** at **115200** baud to see the board's messages.

> **Tip:** Only one Arduino IDE window should be open while compiling. Two windows can delete each other's build files and cause a 'No such file or directory' error.

Serial Monitor commands

Key	Main board	Sensor boards
s	Show status: soil, temperature, forecast, growth stage, pump, safety lock	Show status: raw value, moisture, Wi-Fi
f	Send readings to the database now	Send readings to the database now
w	Test the pump for 15 seconds	-
r	Get the weather forecast now	-
c	Clear the safety lock after checking pump and sensors	-

4. Calibrating the Soil Sensors

Every soil sensor is slightly different. Calibrate each one so 0% means dry air and 100% means water. The raw value goes **down** when the soil gets wetter.

1. Upload the board's sketch and open the Serial Monitor at 115200.
2. Hold the sensor in **dry air**. Send **s** and write down the raw value (usually 2500-3500).
3. Put the sensor in a glass of **water** up to the line. Send **s** and write down the raw value (usually 1000-1500).
4. In that board's sketch, set **SOIL_DRY_RAW** to the dry-air value and **SOIL_WET_RAW** to the water value. Upload again.

```
const int SOIL_DRY_RAW = 3200;   // 0% moisture  (dry air)
const int SOIL_WET_RAW = 1300;   // 100% moisture (in water)
```

> **Note:** If a sensor keeps showing exactly 100% in damp soil, its raw value is below SOIL_WET_RAW. Repeat the water step and use the new value.

What the raw value tells you

Raw value	Meaning
About 1000-3500 and changes with water	Sensor works
Steady 0	No signal: AOUT wire not on D34, no power, or the sensor needs 5 V
Small numbers that jump (e.g. 92, 35, 2, 20)	Nothing is connected to D34 (floating pin)
4000-4095	AOUT is touching 3.3 V, or wired to the wrong pin

To find which pin a sensor is really on, upload **soil_pin_test**. It prints the raw value and millivolts of GPIO 32, 33, 34, 35, 36 and 39 every second.

5. When the Pump Turns On and Off

The main ESP32 checks the rules every second, using the **average soil moisture of the 3 sensors**, the 12-hour rain forecast, the growth stage chosen in the app and, in Vine growth only, the temperature. **Soil** in the tables below means this average.

Sensor 1	Sensor 2	Sensor 3	Average	Pump
50%	80%	85%	(50 + 80 + 85) / 3 = **72%**	OFF (70% or more)
50%	Offline	60%	(50 + 60) / 2 = **55%**	ON (below 60%, rain unlikely)

Vine growth

Soil	Rain chance	Temperature	Pump	Screen
below 60%	below 50%	29 °C or higher	**ON**	IRRIGATE
below 60%	below 50%	below 29 °C	OFF	HOLD
below 60%	50-69%	any	OFF, checks again later	MONITOR
below 60%	70% or higher	any	OFF, rain is coming	RAIN
60-69%	any	any	stays as it is	HOLD
70% or higher	any	any	**OFF**	NO NEED

Flowering and Fruit set (temperature doesn't matter)

Soil	Rain chance	Pump	Screen
below 60%	below 50%	**ON**	IRRIGATE
below 60%	50-69%	OFF, checks again later	MONITOR
below 60%	70% or higher	OFF, rain is coming	RAIN
60-69%	any	stays as it is	HOLD
70% or higher	any	**OFF**	NO NEED

Special cases

- **No temperature reading** (DHT11 problem): Vine growth ignores the 29 °C condition.
- **No rain forecast** (no Wi-Fi, or forecast older than 3 hours): the pump decides on soil only (screen: WATER* when it waters).
- **Rain does not stop a running pump.** Once watering starts it continues until the soil reaches 70%.
- **Offline or faulty sensor:** a sensor with no new reading for 30 seconds, or an invalid reading (raw below 100 or above 4000), is left out of the average.
- **No Wi-Fi:** the main board cannot get sensors 2 and 3, so it uses sensor 1 only.
- **No sensor has a reading:** the pump stays OFF (screen: SENS ERR).
- **5-minute limit:** automatic watering stops after 5 minutes if the soil has not reached 70%, and locks (screen: LOCKED) until you check the pump and send **c** in the Serial Monitor.
- **App buttons:** *Turn pump on* ignores all rules and runs until *Turn pump off* (30-minute safety limit). *Turn pump off* also pauses automatic watering for 1 hour.

> **Caution:** The rain rule uses the forecast, not rain actually falling. If the forecast keeps saying rain but none comes, the soil can keep drying while the pump waits. Watch the soil value in the app on such days.

Where temperature and humidity are used

Place	Temperature	Humidity
Pump (ESP32)	Vine growth only: needs 29 °C or higher	not used
OLED screen	shown as T:30.9C	shown as H:78%
Mobile app	Temperature tile and chart	Humidity tile and chart
Database history	saved with every reading	saved with every reading
ML model	input (about 11% of the decision)	input (about 10% of the decision)

6. The Main Board's Screen (OLED)

Each board shows only its own sensor. The average of the 3 sensors, which the pump uses, is shown only in the mobile app.

```
50%          PUMP OFF       <- sensor 1 (big)  pump state
             NO NEED        <-                 decision code (uses the average)
Soil raw:2150               <- sensor 1 raw value (ERR = bad reading)

T:30.9C  H:78%              <- temperature and humidity (DHT11)
R:100% 5.5mm           WiFi <- rain chance, rainfall (12 h), Wi-Fi
```

Decision codes

Code	Meaning
IRRIGATE	Soil is dry and rain is unlikely: watering
WATER*	Soil is dry, no forecast available: watering on soil only
HOLD	Soil is between 60% and 69%, or (Vine growth) dry but below 29 °C
MONITOR	Soil is dry but rain chance is 50-69%: waiting
RAIN	Soil is dry but rain chance is 70% or more: waiting for the rain
NO NEED	Soil is 70% or more
WATERED	Automatic watering finished (soil reached 70%)
SENS ERR	No soil sensor has a valid reading (all invalid or offline): pump stays off
LOCKED	Safety lock after the 5-minute limit or a sensor error. Check, then send c
PAUSED	Automatic watering paused for 1 hour after Turn pump off in the app
APP ON / APP STOP	Pump turned on / off from the app
APP LIMIT	Pump turned on from the app stopped after 30 minutes
MAX TIME	Automatic watering stopped at the 5-minute limit
TEST / TEST OK	15-second pump test (Serial Monitor w) running / finished

7. The Firebase Database

All data passes through a Firebase Realtime Database. You can view it live in the **Firebase console > Realtime Database > Data**.

Path	Written by	How often	Contents
irrigation/live	Main ESP32	every 2 s	Latest reading: soil (average of the sensors), temperature, humidity, rain forecast, pump state, decision
irrigation/history/...	Main ESP32	every 10 s	Saved copies of the reading (for charts)
irrigation/sensors/soil_1/live	Main ESP32	every 5 s	Sensor 1: soil_moisture_pct, raw, timestamp
irrigation/sensors/soil_2/live	ESP32 #2	every 5 s	Sensor 2 (same fields)
irrigation/sensors/soil_3/live	ESP32 #3	every 5 s	Sensor 3 (same fields)
irrigation/sensors/soil_N/history	each board	every 60 s	Saved sensor readings
irrigation/settings/growth_stage	Mobile app	when you tap a stage	vine_development, flowering or fruiting
irrigation/control	Mobile app	when you tap a pump button	start_request / stop_request (the ESP32 reads and deletes it)
irrigation/prediction	ML service	every 30 s	Latest recommendation
irrigation/prediction_history	ML service	every 30 s	Saved predictions (for your paper)

soil_moisture_pct is left out when a sensor's reading is invalid; only **raw** is saved then. A sensor that has never connected has no entry at all.

Do I need to pay?

No. The project uses Firebase's free **Spark** plan with no credit card. If a limit is reached, Firebase pauses the database until the next month instead of charging you.

Free limit	This system running 24 hours
1 GB stored	About 4 MB per day, about 120 MB per month: fine for about 8 months
100 connections at once	3 boards + ML service + 1-2 phones: fine
10 GB downloaded per month	Fine if the app is opened a few minutes a day. Leaving the app open all day re-downloads the history often and can use up the limit in 2-3 days.

Security: The database currently uses open test rules: anyone who knows its URL can read or change it. Before sharing the project publicly, tighten the rules in Firebase console > Realtime Database > Rules.

8. The ML Model

The model folder is **watermelon_irrigation_ml**. It contains three trained Random Forest models (random-forest-sim-v1):

Model	Answers	Output
irrigation_needed	Should I water now?	Yes/No and a probability
watering_duration	For how long?	Minutes
next_irrigation_hours	When will watering be needed next?	Hours

Inputs: soil moisture, temperature, humidity, rain chance (12 h), forecast rainfall (12 h), hour of the day and growth stage.

Where it runs

The model runs on **this PC**, not in Firebase. Firebase only stores data. The service keeps working only while the PC is on and awake and the program window stays open. When it is not running, the app shows the ESP32's own decision instead.

Starting the model

```
cd C:\Users\chi\Desktop\WaterMelon\watermelon_irrigation_ml
pip install -r requirements.txt          (first time only)
python src/firebase_ml_service.py
```

Each new prediction prints a line such as:

```
[15:46:27] Reading Oct 26 15:46 -> not needed (p=0.00), next in 57.6 h
```

Command	Use
python src/firebase_ml_service.py	Normal use: checks every 30 s. Press Ctrl+C to stop
python src/firebase_ml_service.py --once	Make one prediction and stop (quick test)
python src/firebase_ml_service.py --interval 10	Check every 10 s instead of 30
python src/predict.py sample_mobile_input.json	Test the model offline, without the database

- Set Windows to never sleep while plugged in (Settings > System > Power), and keep the PC time zone on Philippine time (the hour is a model input).
- If a reading is missing a value, the service writes **UNAVAILABLE** instead of guessing.
- The prediction **only advises**. The pump always follows the ESP32 rules in Section 5.

For your paper: The models were trained on simulated field data. Their 'water now' label used the rule soil below 40% and rain chance below 50%, while the ESP32 now waters below 60%. The model and the pump can therefore disagree between 40% and 60% soil.

9. The Mobile App

Starting the app

On this PC (in Chrome):

```
cd C:\Users\chi\Desktop\WaterMelon\watermelon_irrigation_app
flutter run -d chrome
```

On an Android phone: enable USB debugging, connect the phone, then run **flutter run** and choose the phone. The app needs internet to reach the database.

The four tabs

Tab	What you see and do
Home	Recommendation card, soil moisture card with sensors 1-3, temperature and humidity, growth stage buttons, pump card, 12-hour rain forecast
Irrigation	Detailed recommendation (watering minutes, next irrigation) and the inputs it was based on
Analytics	Charts of soil moisture, temperature, humidity, rain forecast and predictions for Today / 7 days / 30 days
Field	Crop, current growth stage and current field conditions

Soil moisture card

The big number is the **average** of the sensors, the value that controls the pump. The label says how many sensors are in it (for example *average of 2 of 3 sensors*). Below it, each sensor has a row:

Row shows	Meaning
A value, e.g. 97%	The sensor is online and its reading is valid
Sensor error	The board is online but the sensor reading is invalid (check wiring)
Offline	No new reading for 30 seconds: board off, wrong Wi-Fi, or never connected

Growth stage

Tap **Vine growth**, **Flowering** or **Fruit set**. The ESP32 picks up the change within 10 seconds. Vine growth adds the 29 °C rule to the pump; the ML model uses the stage for its advice.

Pump card

- **Turn pump on:** the pump runs until you tap **Turn pump off**, whatever the soil reading (30-minute safety limit).
- **Turn pump off:** stops the pump at once and pauses automatic watering for 1 hour.
- If the main ESP32 has sent nothing for 30 seconds, the card shows **Field controller offline**.

Recommendation card

Shows **Irrigation needed** or **Irrigation not needed**, the watering minutes or the time of the next irrigation, and its source: *Estimated by the irrigation model* (ML service running) or the ESP32's decision (ML service not running).

Labels such as **Current**, **Historical**, **Forecast** and **Predicted** show what kind of data each section is. **Historical** means the newest reading is more than 3 hours old.

10. Daily Operation Checklist

1. Power on the three ESP32 boards and the pump supply.
2. On this PC, start the ML service (Section 8) and leave the window open.
3. Open the app. Check that sensors 1-3 show values, not Offline or Sensor error.
4. Check the growth stage is correct for the crop.
5. Look at the recommendation and the 12-hour forecast.
6. Check the main board's screen shows WiFi and a decision code, not SENS ERR or LOCKED.
7. Close the app when you are done, to save the free database download allowance.

Weekly

- Check sensors are still in the soil up to the line and the wires are firm.
- Check the pump intake and hoses are not blocked.
- If readings look wrong (for example always 100%), recalibrate that sensor (Section 4).

11. Troubleshooting

Problem	Likely cause	What to do
Serial: raw=0 OUT OF RANGE; OLED: Soil raw:0 ERR or SENS ERR	Sensor has no power or AOUT is not on D34	Check VCC 3.3 V and AOUT 2.5-3.0 V in air with a multimeter; move AOUT to the D34 row; try VIN (5 V) for NE555 sensors
raw jumps between small numbers (92, 35, 2...)	Nothing connected to D34	Push the yellow wire into the D34 row; replace the jumper
Soil always 100%	Raw value is below SOIL_WET_RAW	Recalibrate (Section 4)
Wi-Fi: disconnected, reason 201	Wi-Fi network not found	Check the network name in secrets.h (same on all boards), 2.4 GHz, router in range
reason 15 or 204	Wrong Wi-Fi password	Fix WIFI_PASS in secrets.h and upload again
Sensor shows Offline in the app	Board off, not on Wi-Fi, or never uploaded	Open its Serial Monitor; look for Wi-Fi connected and Firebase: live updated
Field controller offline (app)	Main ESP32 off or no Wi-Fi	Check power and Wi-Fi of the main board
OLED: LOCKED	5-minute limit or sensor error during watering	Check water supply and sensor, then send c in the Serial Monitor
Recommendation unavailable / old	ML service not running or a value is missing	Start firebase_ml_service.py; fix missing sensor readings
Upload: Failed to write to target RAM (checksum error)	USB data errors	Set Upload Speed to 115200; try another cable; avoid USB hubs
Compile: WiFi.h: No such file or directory	Wrong board selected	Tools > Board > ESP32 Dev Module
Compile: ...ino.cpp: No such file or directory	Two IDE windows sharing a build folder	Close all IDE windows, reopen one, wait 30 s, then Verify
Pump does not start although soil is dry	Rain forecast, Vine growth below 29 °C, PAUSED or LOCKED	Check the OLED decision code (Section 6) and the rules (Section 5)

12. Settings Reference

These values are at the top of **smart_irrigation_esp32.ino**. Change a value, then upload the sketch again.

Setting	Value	Meaning
MOIST_DRY	60	Pump turns on below this soil %
MOIST_WET	70	Pump turns off at this soil % or higher
RAIN_MID	0.50	Rain chance (50%) at which watering waits
RAIN_HIGH	0.70	Rain chance (70%) treated as rain is coming
VINE_TEMP_ON	29.0	Vine growth needs this temperature (°C) or higher
SOIL_DRY_RAW / SOIL_WET_RAW	3200 / 1300	Calibration: raw value at 0% and 100%
RAW_MIN_VALID / RAW_MAX_VALID	100 / 4000	Raw values outside this range = sensor error
PUMP_MAX_RUN_MS	5 min	Longest automatic watering run
APP_MAX_RUN_MS	30 min	Safety limit for Turn pump on from the app
APP_STOP_PAUSE_MS	1 hour	Pause after Turn pump off in the app
MANUAL_TEST_MS	15 s	Pump test (Serial Monitor w)
LIVE_INTERVAL_MS / HISTORY_INTERVAL_MS	2 s / 10 s	Live reading / history record
WEATHER_INTERVAL_MS	30 min	How often the forecast is fetched
FARM_LAT / FARM_LON	6.9636 / 126.0117	Farm position for the forecast
SOIL_PIN, DHT_PIN, RELAY_PIN	34, 4, 26	ESP32 pins
RELAY_ACTIVE_LOW	true	Flip if the pump runs while the log says pump=OFF
DHT_TYPE	DHT11	Change to DHT22 if you use that sensor

Sensor boards (soil_node_2 / soil_node_3) have their own **NODE_ID** (2 or 3), **SOIL_PIN**, calibration values, live interval (5 s) and history interval (60 s).

Project folders

```
WaterMelon/
  smart_irrigation_esp32/    main board sketch (+ secrets.h)
  soil_node_2/               sensor board 2 sketch (+ secrets.h)
  soil_node_3/               sensor board 3 sketch (+ secrets.h)
  soil_pin_test/             wiring test sketch
  watermelon_irrigation_ml/  ML models and Firebase service
  watermelon_irrigation_app/ Flutter mobile app
  PUMP_RULES.txt             pump rules summary
```

> **Security:** secrets.h files hold your Wi-Fi password and API key. They are excluded from GitHub; keep them private.